Problem Description: Container With Most Water

You are given an array h of n non-negative integers, where $h[i]$ represents the height of the i-th vertical rod planted at position i along a horizontal axis. Any two rods, together with the horizontal ground between them, form a **container**. When water is poured into this container, it fills up to the height of the *shorter* of the two chosen rods.

The **volume** of water held by the container formed by rods at indices i and j (with $i < j$) is:

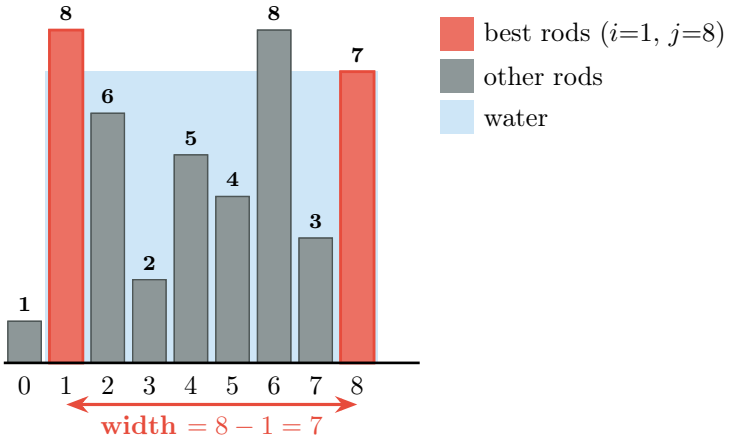
$$V(i,\, j) \;=\; (j - i) \;\times\; \min\big(h[i],\, h[j]\big)$$

where $(j - i)$ is the *width* of the container and $\min(h[i], h[j])$ is the effective *height* (limited by the shorter rod).

Goal: find the pair of indices (i^*, j^*) that maximises $V(i, j)$.

Visual Example

Consider the array $h = [1, 8, 6, 2, 5, 4, 8, 3, 7]$ ($n = 9$ rods, indices 0–8).



The optimal container uses rods at $i = 1$ (height 8) and $j = 8$ (height 7), giving:

$$V(1,\, 8) = (8 - 1) \times \min(8, 7) = 7 \times 7 = \mathbf{49}$$

Problem Statement

Task

Given a list h of $n \geq 2$ non-negative integers, implement two functions:

1. `max_water_iterative(h)` — solve the problem iteratively.
2. `max_water_recursive(h)` — solve the problem using pure recursion (no loops).

Both functions must return a tuple `(volume, i, j)` where `volume` is the maximum water volume and `i`, `j` are the indices of the two chosen rods ($i < j$).

Function signatures

```python
def max_water_iterative(h: list[int]) -> tuple[int, int, int]:
    """
    Find the container with the maximum water volume.

    Parameters
    ----------
    h : list of non-negative ints, length >= 2.

    Returns
    -------
    (volume, i, j) where volume = max water and i < j are the rod
        indices.

    """
    # TODO: implement iterative solution
    pass


def max_water_recursive(h: list[int]) -> tuple[int, int, int]:
    """
    Find the container with maximum water volume using recursion.

    Parameters
    ----------
    h : list of non-negative ints, length >= 2.

    Returns
    -------
    (volume, i, j) where volume = max water and i < j are the rod
        indices.

    """
    # TODO: implement recursive solution
    pass
```

Listing 1: Skeleton — complete the function bodies

Solution: insights

Both solutions rest on the same fundamental observation. Suppose we have two pointers l (left) and r (right) currently pointing at $h[l]$ and $h[r]$.

Claim: if $h[l] \leq h[r]$, then no container that keeps the left rod fixed at l and moves the right rod *inward* to any $r' < r$ can beat the current container. Therefore, we can safely discard rod l and advance $l \leftarrow l + 1$.

This greedy argument is the entire algorithm. Starting from the widest possible container ($l = 0$, $r = n - 1$) and converging inward, we are guaranteed to encounter the optimal pair before l and r cross.

At each step, the pointer on the *shorter* rod advances inward (advance l if $h[l] \leq h[r]$, else advance r). After $n - 1 = 8$ steps, the pointers meet and we return the best volume seen.

Solutions

Iterative Solution (Two Pointers)

```python
def max_water_iterative(h):
    """Time  : O(n)   -- single left-to-right pass with two
       pointers
    Iterative two-pointer solution, not the brute force one.
    """
    l, r = 0, len(h) - 1
    best_vol = 0
    best_i, best_j = l, r

    while l < r:
        vol = (r - l) * min(h[l], h[r])
        if vol > best_vol:
            best_vol, best_i, best_j = vol, l, r

        # Advance the pointer on the shorter rod
        if h[l] <= h[r]:
            l += 1
        else:
            r -= 1

    return best_vol, best_i, best_j
```

Recursive Solution

The recursive version mirrors the iterative logic exactly: the two pointers l and r are passed as function arguments instead of being local loop variables. Each recursive call advances one pointer and keeps a running best.

```python
def _water_helper(h, l, r, best_vol, best_i, best_j):
    """
    Recursive helper. At each call:
      1. Compute V(l, r) and update best if improved.
      2. Advance the shorter pointer and recurse.
    Base case: l >= r  (pointers have crossed -- no valid
       container).

    Time  : O(n)   -- T(n) = T(n-1) + O(1)
    """
    # Base case: pointers have met or crossed
    if l >= r:
        return best_vol, best_i, best_j

    # Evaluate current container
    vol = (r - l) * min(h[l], h[r])
    if vol > best_vol:
        best_vol, best_i, best_j = vol, l, r

    # Recursive step: advance the shorter rod's pointer
    if h[l] <= h[r]:
```

```
        return _water_helper(h, l + 1, r, best_vol, best_i, best_j
            )
    else:
        return _water_helper(h, l, r - 1, best_vol, best_i, best_j
            )


def max_water_recursive(h):
    return _water_helper(h, 0, len(h) - 1, 0, 0, len(h) - 1)
```

The recursive helper is a **tail-recursive** function: the very last operation in every branch is a recursive call, with no further work done on return.

Complexity Analysis

Iterative solution

$\mathcal{O}(n)$: Each iteration advances l or r by exactly one step. The total number of advances before $l \geq r$ is exactly $n - 1$ (they start $n - 1$ apart and converge one step at a time). Each iteration does $\mathcal{O}(1)$ work.

Recursive solution

$\mathcal{O}(n)$: The recurrence is $T(n) = T(n-1) + \mathcal{O}(1)$, with base case $T(0) = \mathcal{O}(1)$. Unrolling: $T(n) = n \cdot \mathcal{O}(1) = \mathcal{O}(n)$. Equivalently, each call shrinks the active window $(r - l)$ by exactly 1, and there are at most $n - 1$ calls before $l \geq r$.

Comparison with the brute-force approach

Approach	Time	Space	Notes
Brute force (all pairs)	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Two nested loops; correct but slow for large n
Iterative two-pointer	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Optimal in both time and space
Recursive two-pointer	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Same time as iterative; tail-recursive but no TCO in Python

For $n = 10^5$ the brute-force requires $\approx 5 \times 10^9$ operations; the two-pointer approaches require $\approx 10^5$ — a factor of 50,000× speedup.

Test Cases

Test	Description	Expected volume	Expected pair
Standard example	$h = [1, 8, 6, 2, 5, 4, 8, 3, 7]$	49	(1, 8)
Two rods	$h = [1, 1]$	1	(0, 1)
Ascending	$h = [1, 2, 3, 4, 5]$	6	(1, 4) or (0, 4)*
Descending	$h = [5, 4, 3, 2, 1]$	6	(0, 3) or (0, 4)*
Plateau	$h = [5, 5, 5, 5]$	15	(0, 3)
Single spike	$h = [1, 1, 1, 100, 1, 1, 1]$	6	(0, 6)
All equal	$h = [3, 3, 3, 3, 3]$	12	(0, 4)
Large gap	$h = [10, 1, 1, 1, 1, 1, 10]$	60	(0, 6)

* Multiple pairs may achieve the same volume; any correct maximum is acceptable.